

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192521313
Name	Vaishnavi.S

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Vaishnavi.S (192521313) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: vaishnavi.s

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and	Most constraints identified	Some constraints identified	Constraints poorly understood

		prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Question 1

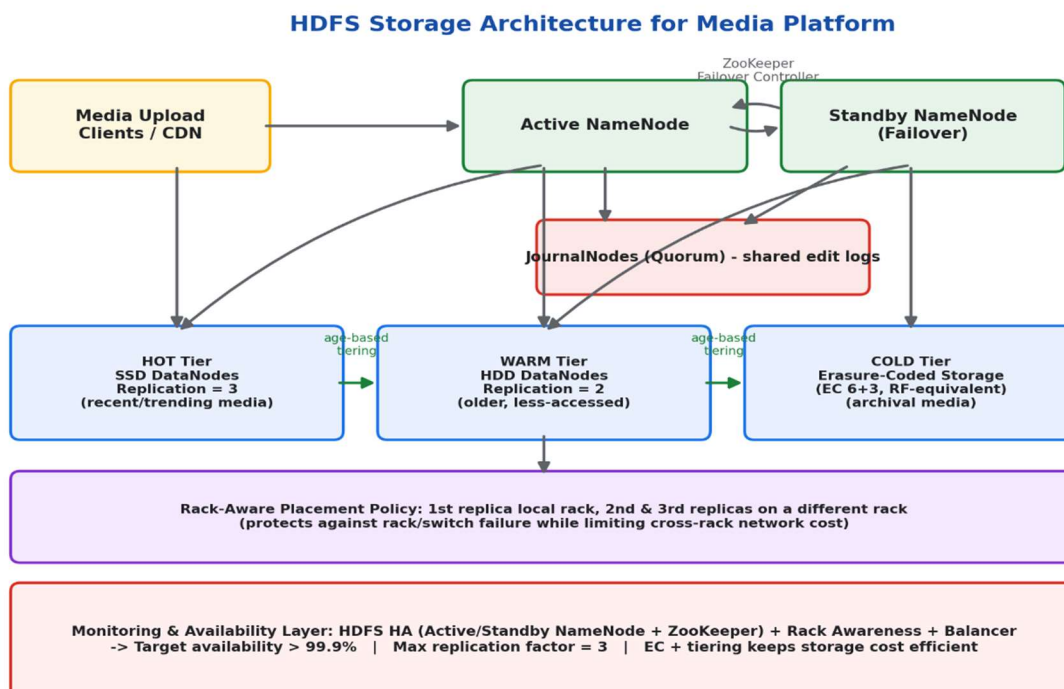
Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

1. Constraints Identified & Prioritized

- **Constraint A (Hard SLA):** Availability $> 99.9\%$ (≈ 8.76 hours downtime/year) — highest priority because the platform is user-facing and any NameNode or DataNode outage directly stops uploads/streaming.
- **Constraint B (Hard limit):** Replication factor ≤ 3 — a hard ceiling; availability and durability cannot simply be bought with more replicas.
- **Constraint C (Cost):** Cost-efficient storage growth — media (video/image) volume grows continuously, so raw $3\times$ replication on all data is economically unsustainable long term.

2. System Design

The proposed system uses an HDFS-based distributed storage architecture where media files such as videos, images, and audio are collected through a data ingestion layer and stored across multiple DataNodes. The NameNode manages file metadata and block locations, while a Standby NameNode provides high availability and failover. A replication factor of 3 is used, with replicas distributed across different racks using rack awareness to protect against node and rack failures. Spark or MapReduce can be used for large-scale media processing and analytics. To maintain cost efficiency, old or rarely accessed media can be moved to low-cost archival storage, while new DataNodes can be added as the media platform grows.



Metadata / Control Plane

- An Active–Standby NameNode pair, coordinated by ZooKeeper Failover Controllers, removes the single point of failure that traditionally caps HDFS availability.
- A JournalNode quorum stores the shared edit log so the Standby NameNode can take over within seconds of an Active failure, with no data loss (fencing prevents split-brain).

Data Plane — Tiered Storage Policies

- **Hot:** HOT tier (recently uploaded / trending media): SSD-backed DataNodes, replication factor = 3 for maximum read throughput and fault tolerance where it matters most.
- **Warm:** WARM tier (older, less-accessed media): HDD-backed DataNodes, replication factor = 2, automatically demoted by HDFS Storage Policies + the Mover tool based on access age.
- **Cold:** COLD tier (archival media): Erasure Coding (e.g., RS-6-3) instead of full replication — it gives fault tolerance equivalent to triple replication at roughly half the storage overhead.
- **Placement policy:** Rack-aware block placement keeps the first replica local and places the remaining replicas on a different rack, protecting against rack/switch failure without exceeding RF = 3.

3. Application of Hadoop Concepts

The design directly applies HDFS HA (NameNode/JournalNode/ZooKeeper), the default rack-aware Block Placement Policy, Heterogeneous Storage + Storage Policies (HOT/WARM/COLD), the Mover/Balancer utilities for lifecycle automation, and Erasure Coding as an HDFS-native alternative to replication.

4. Justification

The proposed design uses NameNode High Availability, 3-way replication, and rack-aware placement to maintain data availability above 99.9%. The replication factor of **3** provides fault tolerance without violating the given constraint. HDFS is suitable for a media platform because it can store and process very large files across multiple DataNodes. Compression, suitable block sizes, and archival of inactive data help control storage costs. Therefore, the design provides a good balance of availability, reliability, scalability, and cost efficiency.

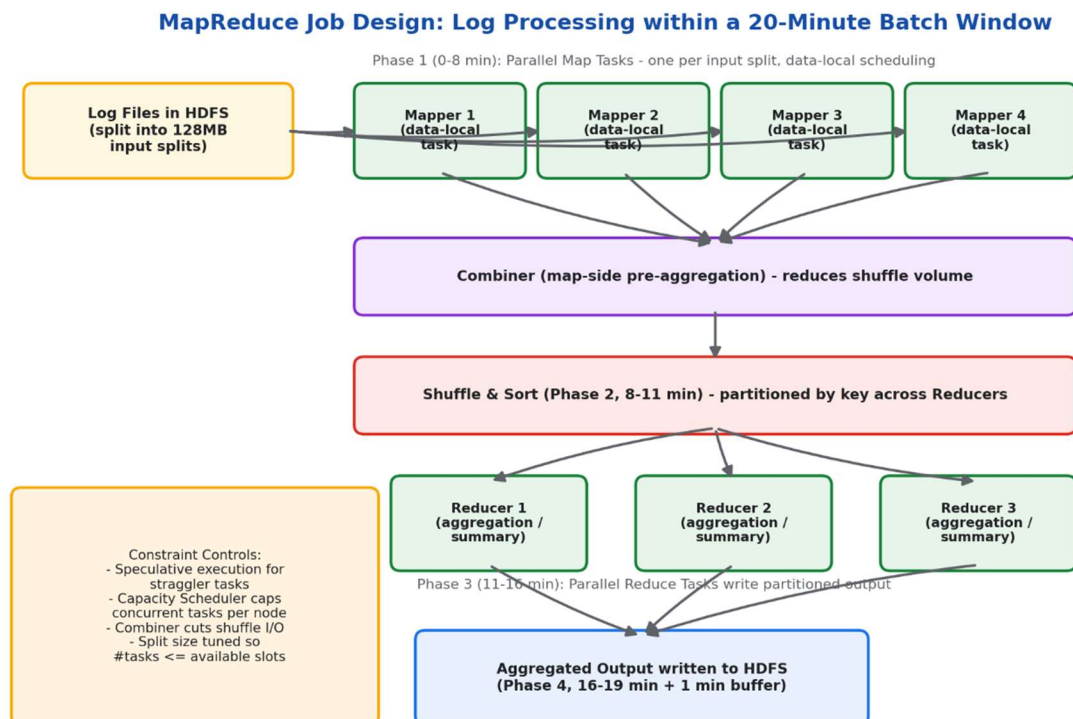
Question 2

A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

- **Constraint A (Time):** Hard time SLA — the entire MapReduce job, including map, shuffle/sort, reduce, and output write, must finish within the 20-minute batch window.
- **Constraint B (Resources):** Limited cluster nodes — parallelism is restricted, so the design must maximize CPU, memory, and disk utilization on the available nodes.
- **Constraint C (Data Volume):** Large log datasets — log files must be divided into appropriate input splits so that multiple Map tasks can process them in parallel.
- **Constraint D (Load Balance):** Uneven task workloads should be avoided — Mapper and Reducer tasks must be distributed evenly to prevent bottlenecks and delays.
- **Constraint E (Network Overhead):** Shuffle traffic must be minimized — data locality and efficient partitioning should be used to reduce unnecessary data movement.

2. System Design & Task Distribution

The log data is first stored in HDFS and divided into suitable input splits. Each split is assigned to a Map task, and the Map tasks are distributed across the limited cluster nodes for parallel processing. Data locality is used to run tasks near the data, reducing network traffic. The Mapper output passes through the shuffle and sort phase, where intermediate data is grouped and distributed to the appropriate Reducers. Reduce tasks are then balanced across the available nodes to avoid bottlenecks. Finally, the Reducers write the processed results back to HDFS. This design uses the available resources efficiently and helps complete the entire MapReduce job within the 20-minute SLA.



Phase-wise Time Budget

- Phase 1 — Map (0–8 min): log file is split into ~128 MB input splits, sized so the number of map tasks equals (or slightly exceeds) the number of available map slots — this avoids a second wave of scheduling delay.
- Phase 2 — Shuffle & Sort (8–11 min): a Combiner performs map-side pre-aggregation so far less intermediate data crosses the network; intermediate output is compressed (e.g., Snappy) to shrink shuffle I/O further.
- Phase 3 — Reduce (11–16 min): the number of reducers is capped to match available reduce slots so tasks run in one wave; partitioning is key-based for balanced load across reducers.
- Phase 4 — Output write + buffer (16–20 min): aggregated results are written back to HDFS with 4 minutes reserved as a safety buffer for stragglers.

3. Application of Hadoop Concepts

The solution applies HDFS to store large log files in distributed blocks across cluster nodes. MapReduce is used to process the logs in parallel, where Mapper tasks read and filter log records and Reducer tasks aggregate the required results. Data locality reduces network traffic by processing data close to where it is stored. Shuffle and Sort groups intermediate key-value pairs before sending them to Reducers. Task parallelism and load balancing ensure efficient use of the limited cluster nodes, while fault tolerance allows failed tasks to be re-executed. These Hadoop concepts help the job meet the 20-minute processing requirement.

4. Justification

- Matching the number of input splits to the available task slots maximizes parallel execution and avoids unnecessary second-wave scheduling, helping the job stay within the 20-minute SLA.
- Using a Combiner and compressing intermediate Mapper output reduces shuffle data, minimizing network and disk overhead during the shuffle/sort phase.
- Using speculative execution helps handle slow or failed tasks by running duplicate tasks on other available slots, reducing the impact of stragglers without adding new nodes.
- Balancing Mapper and Reducer workloads prevents individual nodes from becoming bottlenecks and improves overall cluster utilization.
- Reserving a small buffer within the 20-minute window for shuffle delays and output writing provides additional protection against unexpected processing delays.

Question 3

A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

1. Constraints Identified & Prioritized

Constraint A (Fairness): Cluster resources must be distributed fairly among ETL, reporting, and machine learning jobs so that no workload is completely starved.

- Constraint B (Priority): ETL jobs must receive higher scheduling priority because they are generally required to prepare data for downstream reporting and machine learning workloads.
- Constraint C (Resource Sharing): Limited cluster resources must be divided efficiently among different applications without allowing one workload to consume the entire cluster.
- Constraint D (Failure Recovery): The scheduler must recover jobs when a NodeManager or worker node fails, without requiring manual job restart.

2. System Design

Use the YARN Capacity Scheduler with separate queues for ETL, reporting, and machine learning workloads. The ETL queue is assigned higher priority and a larger guaranteed capacity, while reporting and machine learning receive fair minimum resource allocations. Within each queue, resources are shared fairly among running applications. The ResourceManager monitors the cluster and allocates containers to NodeManagers based on available CPU and memory. If a node fails, YARN detects the failure and reschedules the affected tasks on healthy nodes.

Queue Configuration

- ETL Queue — guaranteed capacity 50%, max capacity 70%, priority HIGH, preemption enabled over other queues so ETL SLA is met even under contention.
- Reporting Queue — guaranteed capacity 25%, max capacity 45%, priority MEDIUM, jobs ordered by the Fair policy so simultaneous reports share resources evenly.
- ML Training Queue — guaranteed capacity 25%, max capacity 45%, priority LOW / elastic, allowed to opportunistically use spare cluster capacity when ETL and Reporting are idle.

Node-Failure Recovery

- The ResourceManager runs as an Active–Standby HA pair coordinated through ZooKeeper, so scheduler state itself survives an RM crash.
- On a NodeManager heartbeat timeout, the ResourceManager marks the node lost, and all containers on that node are rescheduled onto healthy nodes within each queue's capacity limits.
- The affected ApplicationMaster requests replacement containers; if capacity is tight, ETL containers are permitted to preempt lower-priority Reporting/ML containers to protect the ETL SLA, and preempted jobs resume once capacity is restored.

3. Application of Hadoop Concepts

The design uses YARN ResourceManager for global resource management and NodeManagers for monitoring and executing containers. Capacity Scheduler provides queue-based resource allocation and supports ETL priority. Fair sharing within queues prevents reporting and machine learning jobs from being starved. Container allocation ensures CPU and memory are controlled for each application. YARN's failure detection and task re-execution provide recovery when a node fails.

4. Justification

Giving ETL jobs higher queue priority ensures critical data-processing tasks receive resources quickly without completely blocking other workloads.

- Using separate queues provides controlled resource sharing between ETL, reporting, and machine learning jobs.
- Fair scheduling within queues prevents lower-priority reporting and ML jobs from being permanently starved.
- Setting minimum and maximum queue capacities prevents one workload from consuming the entire cluster.
- Automatic task re-execution on healthy nodes allows jobs to recover from NodeManager or worker-node failures without manual intervention.
- The policy balances priority, fairness, resource utilization, and fault recovery within the YARN-managed cluster.

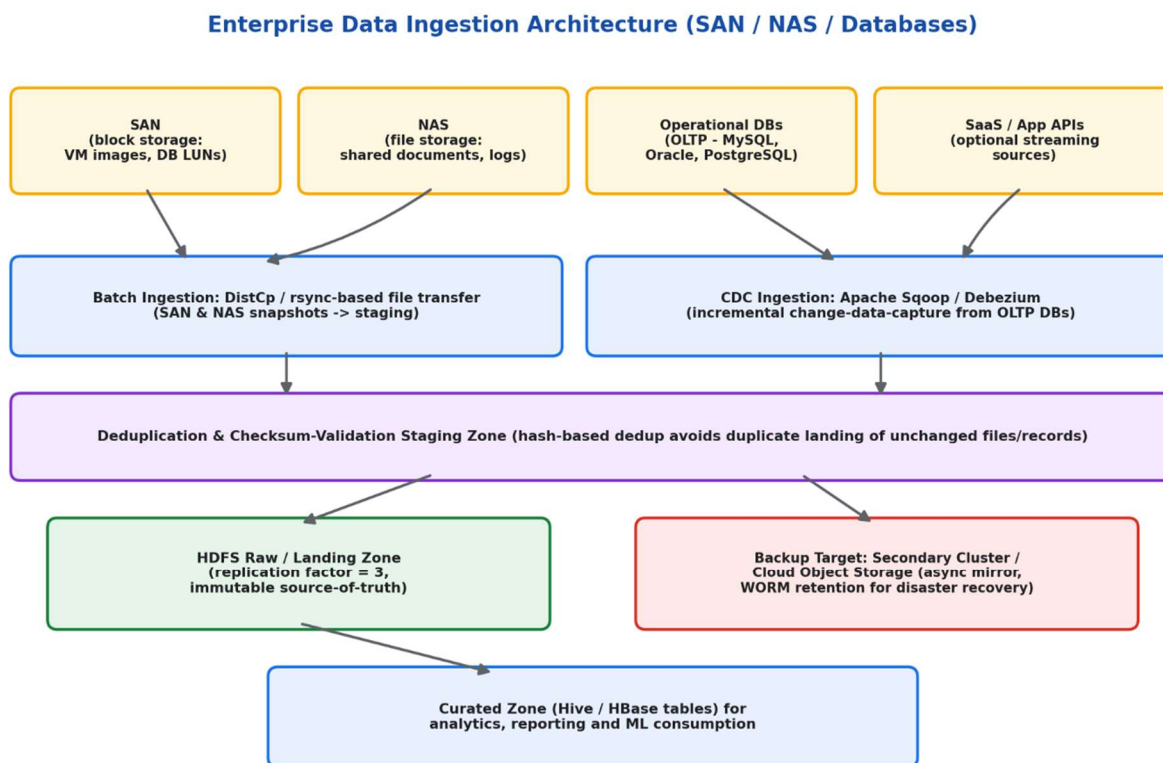
Question 4

An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

1. Constraints Identified & Prioritized

- **Constraint A:** Minimal duplication — the same file or record must not be repeatedly re-copied into the platform once it is already present and unchanged.
- **Constraint B:** Backup reliability — the platform must guarantee a durable, restorable copy of ingested data independent of the primary ingestion pipeline.
- **Constraint C:** Heterogeneous sources — SAN (block storage), NAS (file storage), and operational databases (OLTP) each need a different, appropriate ingestion mechanism.

2. Constrained Architecture



Design meets constraints: minimal duplication (dedup + CDC instead of full re-copy), backup reliability (async DR mirror), auditable lineage

Figure 4: Enterprise ingestion architecture for SAN, NAS, and operational databases

Component-by-Component Design

- SAN / NAS → Batch Ingestion (DistCp / rsync-based transfer): snapshots of block and file storage are pulled into a staging area using parallel, checksum-verified copy tools.

- Operational DBs → CDC Ingestion (Apache Sqoop for scheduled pulls, Debezium for log-based Change-Data-Capture): only inserted/updated/deleted rows since the last run are captured, instead of re-exporting entire tables.
- Deduplication & Checksum-Validation Staging Zone: incoming files/records are hashed and compared against existing landing-zone content; unchanged data is skipped rather than re-landed.
- HDFS Raw/Landing Zone: the deduplicated data is written once with replication factor 3, forming an immutable source-of-truth.
- Backup Target (secondary cluster or cloud object storage): an asynchronous mirror of the landing zone, with WORM (write-once-read-many) retention, dedicated purely to disaster recovery.
- Curated Zone (Hive/HBase tables): cleaned, query-ready data for analytics, reporting, and ML consumption.

3. Application of Hadoop Concepts

The architecture uses DistCp for scalable parallel bulk copy, Sqoop/Debezium for database ingestion via CDC, HDFS's default 3x replication and rack awareness in the landing zone, and a Hive/HBase curated layer as the standard Hadoop-ecosystem consumption pattern.

4. Justification

- CDC ingestion (rather than full re-extraction) is what actually satisfies the minimal-duplication constraint for databases: only deltas move, cutting both duplicate storage and load on production OLTP systems.
- The hash-based dedup/checksum stage catches the SAN/NAS case — repeated snapshots of largely unchanged file shares would otherwise re-land identical objects.
- Making the backup an asynchronous mirror (not a synchronous write) decouples backup latency from ingestion SLA, so backup reliability is achieved without slowing the primary pipeline — and WORM retention protects the backup from accidental or malicious overwrite.
- Keeping the landing zone immutable and separate from the curated zone preserves an auditable, replayable source-of-truth if downstream transformations ever need to be re-run.

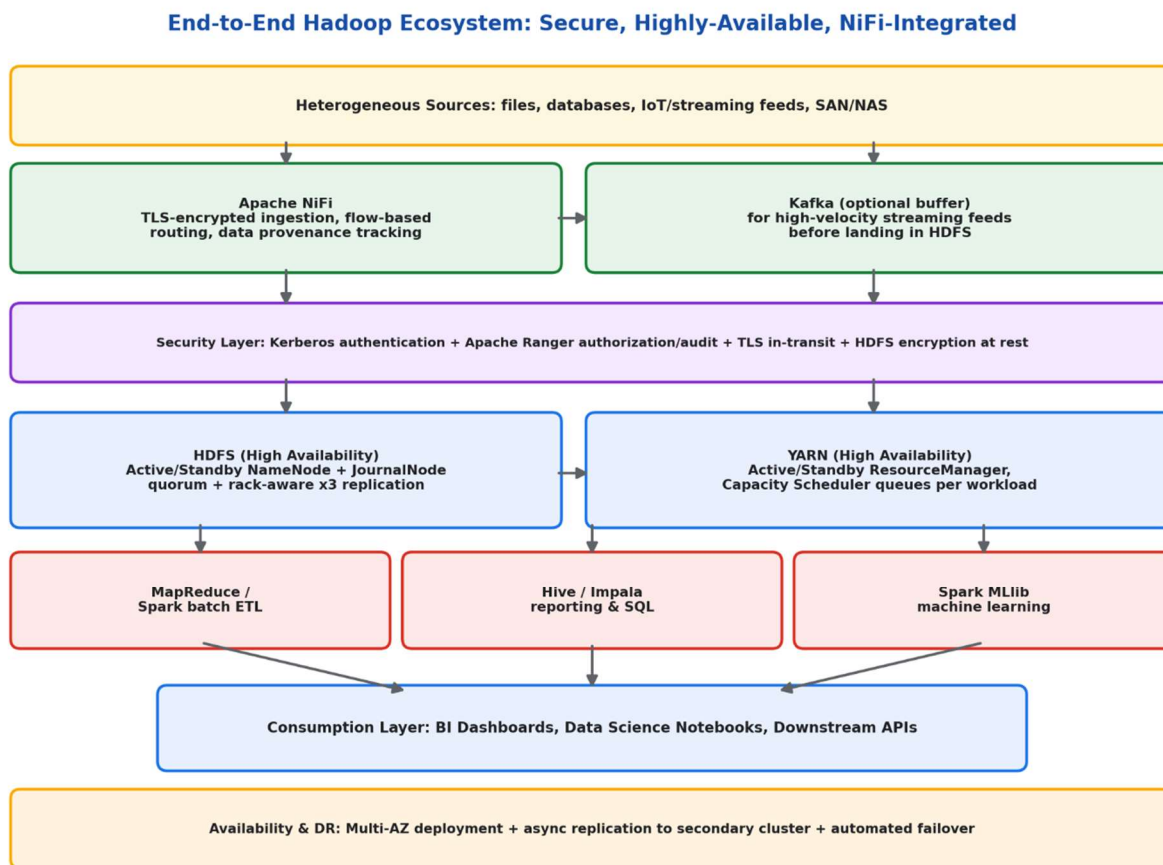
Question 5

Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

1. Constraints Identified & Prioritized

- Constraint A (Security): All data movement between source systems, NiFi, and Hadoop must be secured using encryption, authentication, and access control.
- Constraint B (Availability): Hadoop services must remain available even when individual nodes or services fail, requiring redundancy and automatic failover.
- Constraint C (Integration): Apache NiFi or ETL pipelines must reliably ingest, transform, and route data into the Hadoop ecosystem.
- Constraint D (Scalability): The architecture should support increasing data volumes without major redesign.

2. End-to-End System Design



Layered Design

- Ingestion: Apache NiFi ingests from files, databases, IoT/streaming feeds, and SAN/NAS over TLS, with every flow's lineage tracked via NiFi's built-in data provenance; a Kafka buffer optionally absorbs high-velocity streams before they land in HDFS.
- Security layer: Kerberos handles cluster-wide authentication, Apache Ranger provides fine-grained authorization and audit logging across HDFS/Hive/YARN, TLS protects all in-transit traffic, and HDFS Transparent Data Encryption protects data at rest.
- Storage & resource layer: HDFS runs with Active/Standby NameNodes and a JournalNode quorum (RF = 3, rack-aware); YARN runs with Active/Standby ResourceManagers and Capacity Scheduler queues per workload.
- Processing layer: MapReduce/Spark for batch ETL, Hive/Impala for SQL-based reporting, and Spark MLlib for machine learning — all reading from the same secured, highly-available HDFS.
- Consumption layer: BI dashboards, data-science notebooks, and downstream APIs consume curated output.
- Availability & DR: multi-AZ cluster deployment with asynchronous replication to a secondary cluster and automated failover.

3. Application of Hadoop Concepts

The design uses HDFS for distributed and fault-tolerant storage, YARN for cluster resource management, and Spark/MapReduce for distributed processing. HDFS replication protects data from node failures. NameNode High Availability maintains access to the file system during NameNode failure. NiFi provides flow-based data ingestion, routing, transformation, monitoring, and back-pressure control. Hive can provide SQL-based analysis of the stored data.

4. How the Design Satisfies Every Constraint

- Secure data movement: TLS encrypts every hop from source to NiFi to HDFS; Kerberos ensures only authenticated principals can move data; Ranger enforces who is authorized to read/write each dataset and logs every access; HDFS TDE keeps data encrypted once it is at rest — together this secures the entire data movement path, not just the network link.
- High availability: HA NameNode/ResourceManager pairs plus JournalNode/ZooKeeper quorum remove metadata- and scheduling-layer single points of failure; multi-AZ deployment with async DR replication protects against a full-site outage, satisfying availability at both the component and the site level.
- NiFi/ETL integration: NiFi's flow-based routing and data provenance give an auditable, low-code ingestion layer that lands data directly into the secured, highly-available HDFS zone, from which Spark/Hive/MLlib pipelines consume it immediately — so ingestion and processing are integrated rather than bolted together.

Justification

- Encrypting data during transfer between sources, NiFi, and Hadoop protects sensitive information from unauthorized access.
- Authentication and role-based access control ensure that only authorized users and services can access Hadoop resources.

- HDFS replication provides fault tolerance by maintaining multiple copies of important data.
- NameNode High Availability prevents a single NameNode failure from making the entire HDFS unavailable.
- NiFi provides reliable data ingestion with monitoring, queue management, back-pressure, and controlled data flow